PYTHON

1. Introduction to Python

Introduction

Python is a high-level, interpreted, general-purpose programming language that emphasizes simplicity, readability, and efficiency. It was created by **Guido van Rossum** and officially released in **1991**. Python was designed with the philosophy that code should be easy to read and write, enabling developers to focus more on problem-solving rather than complex syntax.

Over the years, Python has evolved into one of the most widely used programming languages across academia, research, startups, and large-scale industries. Its ability to support rapid development, combined with powerful libraries and frameworks, makes Python an ideal language for beginners as well as experienced programmers.

History and Evolution of Python

Python was developed at the **Centrum Wiskunde & Informatica (CWI)** in the Netherlands as a successor to the ABC programming language. The language name "Python" was inspired by the British comedy series *Monty Python's Flying Circus*, reflecting the creator's intent to make programming enjoyable.

Major milestones in Python's evolution include:

- **Python 1.x** – Basic language features
- **Python 2.x** – Enhanced performance and library support
- **Python 3.x** – Improved Unicode support, cleaner syntax, and better consistency

Python 3 is the current standard and is actively maintained and enhanced by the global Python community.

Characteristics and Features of Python

Python provides a wide range of features that distinguish it from other programming languages:

- **High-Level Language:** Python abstracts complex hardware-level details, allowing programmers to write programs using human-readable syntax.

- **Interpreted Language:** Python code is executed line by line by the Python interpreter, which simplifies debugging and testing.

PYTHON

- **Dynamically Typed:** Variable types are determined at runtime, eliminating the need for explicit type declarations.

- **Object-Oriented and Multi-Paradigm**
  Python supports:

    1. Procedural programming
    2. Object-Oriented programming
    3. Functional programming

- **Platform Independent:** Python programs can run on multiple operating systems without modification.

- **Extensive Standard Library:** Python includes a rich standard library that supports file handling, networking, databases, regular expressions, and more.

- **Open Source and Community Support:** Python is free and open-source, supported by a vast global community that contributes libraries, tools, and documentation.

Applications of Python

Python is used in a wide range of domains:

- **Web Development:** Frameworks such as **Django** and **Flask** are used to develop secure and scalable web applications.

- **Data Science and Analytics:** Python is extensively used for data analysis, visualization, and statistical computing using libraries like **NumPy**, **Pandas**, and **Matplotlib**.

- **Artificial Intelligence and Machine Learning:** Python dominates AI and ML development through frameworks such as **TensorFlow**, **PyTorch**, and **Scikit-learn**.

- **Automation and Scripting:** Python automates repetitive tasks such as file handling, system monitoring, and testing.

- **Internet of Things (IoT):** Python is used in embedded systems and IoT applications using platforms like Raspberry Pi and MicroPython.

- **Cybersecurity:** Python is widely used in penetration testing, vulnerability assessment, and security tool development.

- **Scientific and Engineering Applications:** Python supports scientific simulations, numerical computations, and research applications.

PYTHON

Installing Python and Environment Setup

Python Installation

To begin Python programming, the Python interpreter must be installed:

1. Download Python from the official website.
2. Select the appropriate version for the operating system.
3. Enable **Add Python to PATH** during installation.
4. Verify installation using:
5. python --version

Python Development Environments

IDLE (Integrated Development and Learning Environment)

IDLE is the default IDE that comes with Python installation and provides:

- Interactive Python shell
- Simple text editor
- Debugging support

It is suitable for beginners and educational purposes.

Visual Studio Code (VS Code)

VS Code is a modern and powerful code editor with Python support through extensions. Features include:

- Syntax highlighting
- Code auto-completion
- Integrated debugging
- Version control support

Writing and Executing Python Programs

Python programs can be executed in two modes:

- **Interactive Mode**

PYTHON

Used for quick testing and learning.

```
print("Hello Python")
```

- **Script Mode**

  Programs are saved with .py extension.

```
# sample.py
print("Python Programming")
```

Execution:

```
python sample.py
```

Python Syntax and Indentation

Python uses indentation to define code blocks, making programs visually clean and structured.

Example:

```
if 20 > 10:
    print("Condition is true")
```

Key Syntax Rules:

- Indentation is mandatory
- Python is case-sensitive
- Comments start with #
- Blocks are defined by indentation, not braces

PYTHON

Advantages of Python

- Easy to learn and use
- Faster development time
- Strong ecosystem of libraries
- High demand in industry
- Suitable for beginners and professionals

Python is a powerful, flexible, and versatile programming language that plays a crucial role in modern software development. Its simplicity, combined with extensive functionality, makes it an excellent choice for learning programming fundamentals as well as building advanced applications. A strong understanding of Python basics forms the foundation for advanced topics such as data science, artificial intelligence, and web development.

PYTHON

2. Python Essentials

Overview

Python Essentials form the core foundation of Python programming. This section introduces the fundamental building blocks required to write meaningful and logical Python programs. It covers variables, data types, input/output operations, operators, and coding best practices. Mastering these essentials is crucial for understanding advanced concepts such as control flow, functions, and data structures.

Variables in Python

A **variable** is a name that refers to a memory location used to store data. Python does not require explicit declaration of variables; they are created automatically when a value is assigned.

Characteristics of Python Variables:

- No need to declare data type explicitly
- Case-sensitive
- Can be reassigned to different data types

Example:

```
x = 10
name = "Python"
pi = 3.14
```

Variable Naming Rules:

- Must begin with a letter or underscore (_)
- Cannot start with a number
- Can contain letters, numbers, and underscores
- Cannot use reserved keywords

Data Types in Python

Data types define the kind of data a variable can hold. Python provides several built-in data types.

PYTHON

Common Built-in Data Types:

- **int** – Integer values (e.g., 10, -5)
- **float** – Decimal values (e.g., 3.14)
- **complex** – Complex numbers (e.g., 2+3j)
- **str** – Text or character sequences
- **bool** – Boolean values (True, False)
- **NoneType** – Represents the absence of value

Example:

```
a = 10          # int
b = 2.5         # float
c = "Hello"     # string
d = True        # Boolean
```

Type checking can be done using the type( ) function:

```
print(type(a))
```

Type Conversion (Type Casting)

Python allows conversion between data types using built-in functions.

Common Type Casting Functions:

- int( )
- float( )
- str( )
- bool( )

Example:

```
x = "100"
y = int(x)
```

PYTHON

Input and Output Operations

Input Operation

The input( ) function is used to accept user input. By default, input is read as a string.

Example:

```
name = input("Enter your name: ")
```

To convert input into other data types:

```
age = int(input("Enter your age: "))
```

Output Operation

The print( ) function is used to display output.

Example:

```
print("Welcome to Python")
```

Multiple values can be printed:

```
print("Age:", age)
```

Operators in Python

Operators are symbols used to perform operations on variables and values.

Arithmetic Operators

Used for mathematical calculations.

PYTHON

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
**	Exponentiation
//	Floor division

Example:

```
a = 10
b = 3
print(a + b)
```

Comparison Operators

Used to compare values.

Operator	Description
==	Equal
!=	Not equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

Example:

```
print(a > b)
```

PYTHON

Logical Operators

Used to combine conditional statements.

Operator	Description
and	Logical AND
or	Logical OR
not	Logical NOT

Example:

```
print(a > 5 and b < 5)
```

Assignment Operators

Used to assign values to variables.

Operator	Example
=	x = 10
+=	x += 5
-=	x -= 3
*=	x *= 2

Comments and Code Readability

Comments are used to explain code and improve readability. Python ignores comments during execution.

Single-line Comment:

```
# This is a comment
```

PYTHON

Multi-line Comment (Docstring):

```
"""
This is a multi-line comment
used for documentation
"""
```

Importance of Code Readability:

- Improves maintainability
- Enhances collaboration
- Reduces errors
- Follows Python's philosophy: *"Readability counts"*

Best Coding Practices in Python

- Use meaningful variable names
- Maintain proper indentation
- Write comments where necessary
- Follow PEP 8 coding standards
- Avoid hard-coded values

Python Essentials provide the fundamental knowledge required to write basic programs and understand how Python handles data and operations. Concepts such as variables, data types, input/output mechanisms, and operators form the backbone of Python programming and prepare learners for more advanced topics like control flow, functions, and data structures.

PYTHON

3. Control Flow

Overview

Control Flow refers to the order in which statements in a program are executed. In Python, control flow statements allow the program to make decisions, repeat tasks, and control execution based on conditions. This section introduces conditional statements, looping constructs, loop control statements, and pattern matching using match-case.

Mastering control flow is essential for building logical, efficient, and dynamic Python applications.

Conditional Statements

Conditional statements enable decision-making in Python by executing different blocks of code based on given conditions.

if Statement

The if statement executes a block of code only if the specified condition evaluates to True.

Syntax:

```
if condition:
    statements
```

Example:

```
x = 10
if x > 5:
    print("x is greater than 5")
```

if-else Statement

The if-else statement provides an alternative block of code if the condition is False.

Example:

```
x = 3
if x % 2 == 0:
    print("Even number")
else:
```

```
    print("Odd number")
```

if-elif-else Statement

Used when multiple conditions need to be checked.

Example:

```
marks = 75
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 60:
    print("Grade C")
else:
    print("Fail")
```

Looping Statements

Loops are used to execute a block of code repeatedly until a condition is met.

for Loop

The for loop is used to iterate over a sequence such as a list, tuple, string, or range.

Syntax:

```
for variable in sequence:
    statements
```

Example:

```
for i in range(5):
    print(i)
```

while Loop

The while loop executes as long as the specified condition remains True.

PYTHON

Example:

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

Loop Control Statements

Loop control statements alter the normal execution flow of loops.

break Statement

Terminates the loop immediately.

Example:

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

continue Statement

Skips the current iteration and moves to the next iteration.

Example:

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

pass Statement

Acts as a placeholder where a statement is syntactically required but no action is needed.

Example:

```
if True:
    pass
```

PYTHON

Nested Control Structures

Control structures can be nested within one another.

Example:

```
for i in range(3):
    for j in range(2):
        print(i, j)
```

Match-Case Statement (Structural Pattern Matching)

Python 3.10 introduced the match-case statement, which is similar to switch-case in other languages. It is used for pattern matching and improves readability in decision-making logic.

Syntax:

```
match variable:
    case value1:
        statements
    case value2:
        statements
    case _:
        statements
```

Example:

```
choice = 2
match choice:
    case 1:
        print("Option One")
    case 2:
        print("Option Two")
    case _:
        print("Invalid Choice")
```

Indentation and Control Flow

Python uses indentation to define blocks of code. Incorrect indentation leads to logical or syntax errors.

PYTHON

Example:

```
if x > 0:
    print("Positive")
    if x % 2 == 0:
        print("Even")
```

Best Practices for Control Flow

- Avoid deeply nested conditions
- Use meaningful condition expressions
- Prefer for loops over while loops when iteration count is known
- Use match-case for better readability when handling multiple options

Control Flow structures form the backbone of Python programming logic. Conditional statements enable decision-making, loops support repetition, and control statements refine loop behavior. Understanding these concepts allows developers to write efficient, readable, and logically sound Python programs.

PYTHON

4. Functions and Modules

Overview

Functions and modules are essential components of structured and modular programming in Python. Functions allow a program to be divided into smaller, reusable blocks of code, while modules enable logical organization of related functions and variables into separate files. Together, they improve code readability, reusability, maintainability, and scalability.

Functions in Python

A **function** is a block of organized, reusable code designed to perform a specific task. Python functions help avoid repetition and make programs easier to understand and debug.

Defining a Function

Functions in Python are defined using the def keyword.

Syntax:

```
def function_name(parameters):
   statements
```

Example:

```
def greet():
    print("Welcome to Python")
```

Calling a Function

Once defined, a function can be executed by calling its name.

Example:

```
greet()
```

PYTHON

Function Parameters and Arguments

Parameters are variables listed in a function definition, while arguments are values passed during function calls.

Required Arguments

Arguments that must be provided in the correct order.

Example:

```
def add(a, b):
    print(a + b)

add(5, 3)
```

Keyword Arguments

Arguments passed using parameter names.

Example:

```
add(b=3, a=5)
```

Default Arguments

Parameters assigned default values.

Example:

```
def greet(name="User"):
    print("Hello", name)

greet()
greet("Python")
```

PYTHON

Variable-Length Arguments

Used when the number of arguments is unknown.

- *args – Non-keyword arguments
- **kwargs – Keyword arguments

Example:

```
def total(*numbers):
    print(sum(numbers))

total(1, 2, 3, 4)
```

Return Statement

The return statement sends a value back to the caller and ends the function execution.

Example:

```
def square(x):
    return x * x

result = square(4)
print(result)
```

Scope of Variables

Scope determines the accessibility of variables.

Types of Scope:

- **Local Scope** – Variables inside a function
- **Global Scope** – Variables declared outside functions

Example:

```
x = 10

def show():
    x = 5
    print(x)
```

PYTHON

```
show()
print(x)
```

Lambda (Anonymous) Functions

Lambda functions are small, anonymous functions defined using the lambda keyword.

Syntax:

```
lambda arguments: expression
```

Example:

```
square = lambda x: x * x
print(square(5))
```

Built-in Functions

Python provides many built-in functions to simplify programming.

Common Built-in Functions:

- print( )
- len( )
- type( )
- input( )
- range( )
- sum( )
- max( )
- min( )

Example:

```
numbers = [1, 2, 3]
print(len(numbers))
```

PYTHON

Modules in Python

A **module** is a file containing Python code such as functions, classes, and variables. Modules help organize code into logical components.

Importing a Module

Example:

```
import math
print(math.sqrt(16))
```

Importing Specific Functions

Example:

```
from math import sqrt
print(sqrt(25))
```

Importing with Alias

Example:

```
import math as m
print(m.pi)
```

User-Defined Modules

Users can create their own modules by saving Python code in a .py file.

Example:

```
# mymodule.py
def message():
    print("This is a user-defined module")
```

PYTHON

Usage:

```
import mymodule
mymodule.message()
```

Packages in Python

A **package** is a collection of related modules stored in a directory. It helps in managing large projects efficiently.

Advantages of Packages:

- Logical organization
- Easier maintenance
- Avoids naming conflicts

Best Practices for Functions and Modules

- Use meaningful function names
- Keep functions short and focused
- Avoid global variables
- Organize code into modules
- Follow PEP 8 standards

Functions and modules enable modular programming in Python. Functions promote code reuse and clarity, while modules and packages help organize large applications. Understanding these concepts is essential for building efficient, maintainable, and scalable Python programs.

PYTHON

5. Data Structures in Python

Overview

Data Structures are specialized formats used to store, organize, and manage data efficiently. Python provides a rich set of built-in data structures that allow programmers to handle large volumes of data with ease and flexibility. Proper understanding of data structures is essential for writing efficient, scalable, and maintainable programs.

This section covers the core Python data structures such as **Lists, Strings, Tuples, Dictionaries, and Sets**, along with their operations and real-world usage.

Lists

A **list** is an ordered, mutable collection of elements. Lists can store heterogeneous data types and allow duplicate values.

Creating Lists

Lists are created using square brackets [ ].

Example:

```
numbers = [1, 2, 3, 4]
mixed = [10, "Python", 3.5, True]
```

Accessing List Elements

List elements are accessed using index values (starting from 0).

```
print(numbers[0])
print(numbers[-1])
```

Manipulating Lists

Common list operations include adding, removing, and modifying elements.

```
numbers.append(5)
numbers.insert(1, 10)
numbers.remove(3)
numbers.pop()
```

PYTHON

List Comprehensions

List comprehensions provide a concise way to create lists.

Syntax:

```
[expression for item in iterable if condition]
```

Example:

```
squares = [x**2 for x in range(5)]
```

Working with Strings

Strings are sequences of characters enclosed within single (' '), double (" "), or triple quotes ("""").

Creating Strings

```
text = "Python Programming"
```

String Concatenation

Combining strings using the + operator.

```
a = "Hello"
b = "World"
print(a + " " + b)
```

Multi-line Strings

```
message = """This is
a multi-line
string"""
```

PYTHON

String Formatting

String formatting allows dynamic insertion of values.

Using f-strings:

```
name = "Python"
print(f"Welcome to {name}")
```

Accessing Characters in a String

```
print(text[0])
print(text[-1])
```

String Slicing

Extracting a portion of a string.

```
print(text[7:])
print(text[0:6])
```

String Methods

Python provides many built-in string methods.

Method	Description
lower( )	Converts to lowercase
upper( )	Converts to uppercase
strip( )	Removes spaces
replace( )	Replaces substring
split( )	Splits string

PYTHON

Example:

```
print(text.upper())
```

Tuples

A **tuple** is an ordered, immutable collection of elements.

Characteristics of Tuples

- Immutable (cannot be modified)
- Faster than lists
- Can contain duplicate values

Creating Tuples

```
t = (1, 2, 3, "Python")
```

Accessing Tuple Elements

```
print(t[1])
```

Dictionaries

A **dictionary** stores data in key-value pairs. Keys must be unique and immutable.

Creating Dictionaries

```
student = {
    "name": "Alex",
    "age": 20,
    "course": "Python"
}
```

PYTHON

Accessing and Modifying Data

```
print(student["name"])
student["age"] = 21
```

Key-Value Pair Manipulations

```
student["grade"] = "A"
student.pop("course")
```

Sets

A **set** is an unordered collection of unique elements.

Creating Sets

```
s = {1, 2, 3, 4}
```

Unique Element Operations

Sets automatically remove duplicates.

```
s = {1, 2, 2, 3}
print(s)
```

Set Operations

```
a = {1, 2, 3}
b = {3, 4, 5}

print(a.union(b))
print(a.intersection(b))
```

PYTHON

```
print(a.difference(b))
```

Comparison of Data Structures

Data Structure	Ordered	Mutable	Duplicates
List	Yes	Yes	Yes
Tuple	Yes	No	Yes
Dictionary	Yes (3.7+)	Yes	Keys No
Set	No	Yes	No

Best Practices

- Use lists for dynamic collections
- Use tuples for fixed data
- Use dictionaries for fast lookup
- Use sets for uniqueness checks
- Choose appropriate data structures for performance

Python data structures provide efficient ways to store and manipulate data. Lists, strings, tuples, dictionaries, and sets each serve specific purposes. Understanding their behavior and operations is fundamental to building efficient, scalable Python applications and solving real-world problems effectively.

PYTHON

6. File Handling in Python

Overview

File handling is an essential aspect of programming that enables a program to store data permanently and retrieve it when required. Python provides built-in functions and methods to create, read, write, and manipulate files efficiently. File handling is widely used in applications such as data processing, logging, configuration management, and report generation.

This section covers file operations, handling different file formats, and exception handling during file operations.

Types of Files in Python

Python mainly works with two types of files:

1. **Text Files** – Store data in readable text format (e.g., .txt, .csv)
2. **Binary Files** – Store data in binary format (e.g., images, audio files)

Opening and Closing Files

The open( ) function is used to open a file.

Syntax:

```
file_object = open("filename", "mode")
```

Common File Modes:

Mode	Description
r	Read
w	Write (overwrites file)
a	Append
r+	Read and write
w+	Write and read

PYTHON

Example:

```
file = open("data.txt", "r")
file.close()
```

Reading Files

Reading Entire File

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

Reading Line by Line

```
file = open("data.txt", "r")
for line in file:
    print(line)
file.close()
```

Using readline() and readlines()

```
file.readline()
file.readlines()
```

Writing to Files

Writing Data

```
file = open("output.txt", "w")
file.write("Python File Handling")
file.close()
```

PYTHON

Appending Data

```
file = open("output.txt", "a")
file.write("\nAppending new data")
file.close()
```

Using with Statement

The with statement automatically closes the file after use, improving safety and readability.

Example:

```
with open("data.txt", "r") as file:
    print(file.read())
```

Working with CSV Files

CSV (Comma-Separated Values) files are widely used for data storage and exchange.

Reading CSV Files

```
import csv

with open("data.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

Writing CSV Files

```
with open("data.csv", "w", newline="") as file:
   writer = csv.writer(file)
   writer.writerow(["Name", "Age"])
   writer.writerow(["Alice", 22])
```

PYTHON

Exception Handling in File Operations

File operations may cause runtime errors such as missing files or permission issues. Python uses exception handling to manage such errors gracefully.

Try-Except Block

```
try:
    file = open("test.txt", "r")
    print(file.read())
except FileNotFoundError:
    print("File not found")
finally:
    file.close()
```

Common File-Related Exceptions:

- FileNotFoundError
- PermissionError
- IOError

Best Practices for File Handling

- Always close files after use
- Prefer with statement
- Handle exceptions properly
- Avoid hard-coded file paths
- Validate file existence before reading

File handling in Python enables programs to store and retrieve data efficiently. Python's built-in file handling mechanisms support various file formats and provide robust error-handling capabilities. Understanding file operations is crucial for building data-driven and real-world Python applications.

PYTHON

7. Object-Oriented Programming (OOP) in Python

Overview

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around **objects** rather than functions or logic. Python fully supports OOP concepts, enabling developers to create modular, reusable, and scalable applications. OOP improves code structure, reduces redundancy, and enhances maintainability.

This section introduces the fundamental concepts of OOP, including classes, objects, constructors, methods, inheritance, and polymorphism.

Introduction to OOP Concepts

The core principles of Object-Oriented Programming include:

- **Class** – A blueprint for creating objects
- **Object** – An instance of a class
- **Encapsulation** – Binding data and methods together
- **Inheritance** – Acquiring properties of another class
- **Polymorphism** – Ability to take multiple forms
- **Abstraction** – Hiding implementation details

Defining Classes and Objects

Class Definition

A class is defined using the class keyword.

Syntax:

```
class ClassName:
    statements
```

Example:

```
class Student:
    pass
```

PYTHON

Creating Objects

Objects are created using the class name followed by parentheses.

Example:

```
s1 = Student()
```

Constructors and Instance Variables

Constructor (__init__ Method)

The constructor is a special method that initializes object data when an object is created.

Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Accessing Instance Variables

```
s = Student("Alex", 20)
print(s.name)
print(s.age)
```

Methods in Classes

Methods are functions defined inside a class that operate on objects.

Example:

```
class Student:
    def display(self):
        print("Student Name:", self.name)
```

PYTHON

Inheritance

Inheritance allows one class to acquire properties and methods of another class.

Types of Inheritance:

- Single Inheritance
- Multiple Inheritance
- Multilevel Inheritance

Example:

```
class Person:
    def speak(self):
        print("Speaking")

class Student(Person):
    pass

s = Student()
s.speak()
```

Polymorphism

Polymorphism allows methods to behave differently based on the object.

Method Overriding

```
class Animal:
   def sound(self):
      print("Animal sound")

class Dog(Animal):
   def sound(self):
      print("Bark")
```

Encapsulation

Encapsulation restricts direct access to data and ensures data security.

PYTHON

Example:

```
class Account:
    def __init__(self):
        self.__balance = 0
```

Abstraction

Abstraction hides implementation details and shows only essential features.

Example:

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

Advantages of OOP

- Modular and reusable code
- Improved readability and maintainability
- Easier debugging and testing
- Real-world problem modeling

Best Practices in OOP

- Use meaningful class and method names
- Follow single responsibility principle
- Avoid deep inheritance hierarchies
- Use encapsulation wisely

Object-Oriented Programming in Python provides a powerful way to design and structure applications. Concepts such as classes, objects, inheritance, and polymorphism enable developers to build scalable, maintainable, and reusable software systems.

PYTHON

8. Python Libraries and Tools

Overview

Python provides a rich ecosystem of libraries and tools that significantly enhance productivity and simplify complex programming tasks. Libraries offer pre-written code for specific functionalities, while tools help in development, testing, visualization, and deployment. This section introduces widely used Python libraries such as **NumPy, Pandas, Matplotlib**, and provides an overview of other essential libraries.

NumPy: Basics and Array Operations

Introduction to NumPy

NumPy (Numerical Python) is a fundamental library used for numerical computing in Python. It provides support for multi-dimensional arrays, mathematical functions, and linear algebra operations.

Creating NumPy Arrays

```
import numpy as np

arr = np.array([1, 2, 3, 4])
```

Array Operations

```
print(arr + 2)
print(arr * 3)
```

Multi-Dimensional Arrays

```
matrix = np.array([[1, 2], [3, 4]])
```

Pandas: DataFrames and Series

Introduction to Pandas

Pandas is a powerful data manipulation and analysis library. It provides data structures such as **Series** and **DataFrame** for handling structured data.

PYTHON

Series

```
import pandas as pd

data = pd.Series([10, 20, 30])
```

DataFrame

```
df = pd.DataFrame({
    "Name": ["Alice", "Bob"],
    "Age": [22, 25]
})
```

Data Manipulation

```
print(df.head())
print(df["Age"].mean())
```

Matplotlib: Basic Plotting

Introduction to Matplotlib

Matplotlib is a visualization library used to create static, animated, and interactive plots.

Creating a Simple Plot

```
import matplotlib.pyplot as plt

x = [1, 2, 3]
y = [2, 4, 6]

plt.plot(x, y)
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.title("Simple Line Plot")
plt.show()
```

PYTHON

Introduction to Other Python Libraries

Requests

Used for sending HTTP requests.

```
import requests
response = requests.get("https://example.com")
print(response.status_code)
```

OS Module

Used to interact with the operating system.

```
import os
print(os.getcwd())
```

datetime Module

Used for date and time operations.

```
import datetime
print(datetime.datetime.now())
```

Python Development Tools

IDEs and Editors

- IDLE
- Visual Studio Code
- PyCharm

Package Management

- pip
- virtual environments

PYTHON

Best Practices for Using Libraries

- Use virtual environments
- Import only required modules
- Keep libraries updated
- Follow official documentation

Python libraries and tools significantly reduce development time and enhance application capabilities. Libraries like NumPy, Pandas, and Matplotlib play a crucial role in data analysis and visualization, while other tools support system interaction and web communication.

PYTHON

9. Error Handling and Debugging in Python

Introduction

Error handling and debugging are critical aspects of software development that ensure reliability, maintainability, and robustness of Python applications. Errors are inevitable during program execution due to invalid inputs, logical mistakes, or unexpected runtime conditions. Python provides a structured and efficient mechanism to detect, handle, and debug errors gracefully, allowing programs to recover from failures without crashing abruptly.

Effective error handling improves user experience, simplifies maintenance, and helps developers identify and resolve issues faster. Debugging, on the other hand, is the systematic process of identifying, analyzing, and fixing defects (bugs) in a program.

Types of Errors in Python

Python errors can be broadly classified into three categories:

Syntax Errors

Syntax errors occur when the Python interpreter encounters code that violates the language's grammatical rules. These errors are detected during compilation and prevent the program from running.

Example:

```
if x > 5
    print("Greater")
```

Reason: Missing colon (:)

Runtime Errors (Exceptions)

Runtime errors occur during program execution even if the syntax is correct. These are also known as **exceptions**.

Common runtime exceptions include:

- ZeroDivisionError
- ValueError
- TypeError
- IndexError

PYTHON

- KeyError
- FileNotFoundError

Example:

```
x = 10 / 0
```

This raises a ZeroDivisionError.

Logical Errors

Logical errors occur when a program executes successfully but produces incorrect results due to faulty logic. These errors are the most difficult to detect because the program does not crash.

Example:

```
# Incorrect logic for average
average = total / (count - 1)
```

Exception Handling in Python

Python provides a structured exception-handling mechanism using the try, except, else, and finally blocks.

The try and except Blocks

The try block contains code that may raise an exception, while the except block handles the exception.

Example:

```
try:
    x = int(input("Enter a number: "))
    result = 10 / x
except ZeroDivisionError:
    print("Division by zero is not allowed")
except ValueError:
    print("Invalid input")
```

PYTHON

Multiple Exception Handling

Python allows handling multiple exceptions separately or together.

```
except (ValueError, TypeError):
    print("Invalid data type")
```

The else Block

The else block executes only if no exception occurs.

```
try:
    x = int(input())
    y = 10 / x
except ZeroDivisionError:
    print("Error occurred")
else:
    print("Result:", y)
```

The finally Block

The finally block executes regardless of whether an exception occurs. It is commonly used for cleanup operations such as closing files or database connections.

```
try:
    file = open("data.txt")
    content = file.read()
except FileNotFoundError:
    print("File not found")
finally:
    file.close()
```

Raising Exceptions

Python allows developers to raise custom exceptions using the raise keyword.

```
age = int(input("Enter age: "))
if age < 18:
    raise ValueError("Age must be 18 or above")
```

Custom exception handling improves program validation and enforces business rules.

PYTHON

Debugging in Python

Debugging is the process of identifying and resolving defects in a program. Python provides several techniques and tools to assist debugging.

Print-Based Debugging

The simplest form of debugging uses print() statements to track variable values and execution flow.

```
print("Value of x:", x)
```

Using the Python Debugger (pdb)

Python includes a built-in debugger called pdb that allows step-by-step execution.

```
import pdb
pdb.set_trace()
```

Key pdb commands:

- n – next line
- s – step into function
- c – continue execution
- q – quit debugger

Debugging Using IDEs

Modern IDEs such as **VS Code, PyCharm, and Spyder** provide graphical debugging tools including:

- Breakpoints
- Variable inspection
- Call stack analysis
- Step execution

These tools significantly reduce debugging time and improve productivity.

PYTHON

Common Debugging Techniques

- Analyzing stack traces
- Isolating problematic code sections
- Testing edge cases
- Writing unit tests
- Reviewing code logic systematically

Best Practices for Error Handling and Debugging

- Handle only expected exceptions
- Avoid using broad except: clauses
- Provide meaningful error messages
- Log errors instead of printing them in production systems
- Combine debugging with unit testing

Error handling and debugging are essential for developing reliable and maintainable Python applications. Python's robust exception-handling framework and powerful debugging tools allow developers to identify errors efficiently and handle unexpected situations gracefully. By following best practices and structured debugging approaches, software quality and system stability can be significantly enhanced.

PYTHON

10. Unit Testing with Python

Introduction

Unit testing is a software testing technique in which individual components (units) of a program are tested independently to verify that they function as expected. In Python, unit testing helps developers detect bugs early, ensure code correctness, and maintain software quality during development and future enhancements.

Python provides a built-in framework called **unittest**, which supports test automation, test organization, and result reporting. Unit testing is a fundamental practice in professional software development and is widely adopted in industry-standard projects.

Importance of Unit Testing

Unit testing offers several benefits:

- Early detection of defects
- Improved code reliability and stability
- Easier debugging and maintenance
- Facilitates refactoring and optimization
- Enhances code documentation and understanding

The unittest Framework

The unittest module is inspired by Java's JUnit and follows an object-oriented testing approach.

Key Components of unittest

- **Test Case**: A single unit of testing
- **Test Suite**: Collection of test cases
- **Test Runner**: Executes test cases and reports results
- **Assertions**: Validate expected outcomes

Organizing Tests with the TestCase Class

To create unit tests, developers define a class that inherits from unittest.TestCase. Each test is written as a method whose name starts with test_.

PYTHON

Example:

```
import unittest

class TestMathOperations(unittest.TestCase):

    def test_addition(self):
        self.assertEqual(2 + 3, 5)

    def test_subtraction(self):
        self.assertEqual(5 - 2, 3)
```

Writing Test Cases

A test case verifies a specific behavior of a function or module.

Example:

```
def multiply(a, b):
    return a * b

class TestMultiply(unittest.TestCase):
    def test_positive_numbers(self):
        self.assertEqual(multiply(2, 3), 6)

    def test_zero(self):
        self.assertEqual(multiply(5, 0), 0)
```

Running unittest Tests

Tests can be executed in multiple ways:

Running from Command Line

```
python -m unittest test_file.py
```

Running from Code

```
if __name__ == "__main__":
```

```
unittest.main()
```

The test runner displays results indicating passed tests, failed tests, and errors.

Exploring Assertion Methods

Assertions are used to compare expected and actual results.

Common Assertion Methods

- assertEqual(a, b)
- assertNotEqual(a, b)
- assertTrue(x)
- assertFalse(x)
- assertIsNone(x)
- assertIn(a, b)
- assertRaises(Exception)

Example:

```
with self.assertRaises(ZeroDivisionError):
    x = 10 / 0
```

Setup and Teardown Methods

The setUp() and tearDown() methods prepare and clean up test environments.

```
class TestExample(unittest.TestCase):

    def setUp(self):
        self.value = 10

    def tearDown(self):
        self.value = None
```

Test Results Interpretation

- . → Test passed

- F → Test failed
- E → Error occurred
- OK → All tests successful

Analyzing test output helps developers quickly identify faulty logic.

Best Practices for Unit Testing

- Write tests for all critical functions
- Keep tests independent and isolated
- Use meaningful test names
- Test both valid and invalid inputs
- Automate testing as part of development workflow

Unit testing with Python ensures code correctness, stability, and long-term maintainability. The unittest framework provides a robust and structured approach to writing, organizing, and executing tests. Incorporating unit testing into the development lifecycle leads to higher-quality software and reduced debugging effort.

PYTHON

11. Logging with Python

Introduction

Logging is an essential aspect of software development that enables developers to record events, errors, warnings, and runtime information while an application is executing. Unlike print statements, logging provides a flexible and configurable mechanism to track application behavior, diagnose issues, and monitor system performance in production environments.

Python provides a built-in **logging** module that supports hierarchical loggers, multiple log levels, and configurable output destinations such as console, files, and remote servers.

Importance of Logging

Logging plays a critical role in professional software systems:

- Helps identify and diagnose runtime errors
- Assists in debugging complex applications
- Enables monitoring and auditing
- Improves maintainability and observability
- Provides insight into application flow and performance

Logging Levels

Python's logging module supports multiple severity levels:

Level	Description
DEBUG	Detailed information for debugging
INFO	General operational messages
WARNING	Indication of potential problems
ERROR	Serious issues that prevent functionality
CRITICAL	Severe errors causing program termination

Python's Logging Module

The logging module is included in Python's standard library and does not require external installation.

PYTHON

Basic Logging Example

```
import logging

logging.basicConfig(level=logging.INFO)
logging.info("Application started")
logging.warning("This is a warning message")
logging.error("An error occurred")
```

Configuring the Logging System

Logging behavior can be customized using basicConfig() or advanced configuration methods.

Logging to a File

```
logging.basicConfig(
    filename='app.log',
    level=logging.DEBUG,
    format='%(asctime)s - %(levelname)s - %(message)s'
)
```

This configuration stores logs in a file with timestamps and severity levels.

Logger Objects

Instead of using the root logger, applications should define their own logger instances.

```
logger = logging.getLogger("MyApp")
logger.setLevel(logging.DEBUG)

logger.debug("Debug message")
logger.info("Info message")
```

Using logger objects improves modularity and control.

Logging Formatters

Formatters define how log messages are structured.

Common Format Attributes

PYTHON

- %(asctime)s – Timestamp
- %(levelname)s – Log level
- %(name)s – Logger name
- %(message)s – Log message

Handlers in Logging

Handlers determine where log messages are sent.

Types of Handlers

- StreamHandler – Console output
- FileHandler – File output
- RotatingFileHandler – File rotation
- SMTPHandler – Email alerts

```
handler = logging.FileHandler("system.log")
logger.addHandler(handler)
```

Common Logging Pitfalls and Solutions

Pitfall	Solution
Using print instead of logging	Use logging levels
Logging sensitive information	Mask confidential data
Excessive logging	Adjust log levels
Hard-coded configurations	Use config files

Best Practices for Logging

- Use appropriate log levels
- Avoid logging passwords or secrets
- Maintain consistent log formats
- Rotate log files regularly
- Use logging instead of print statements

PYTHON

Logging with Python provides a powerful mechanism for tracking application behavior and diagnosing issues efficiently. By using Python's logging module effectively, developers can build robust, maintainable, and production-ready applications.

PYTHON

12. Advanced Topics in Python

Advanced Python concepts enable developers to write efficient, scalable, and high-performance applications. This section introduces functional programming techniques, advanced data handling, automation tools, and professional development practices used in real-world Python projects.

Lambda Functions

Lambda functions are **anonymous, single-expression functions** used for short operations.

Syntax

```
lambda arguments: expression
```

Example

```
square = lambda x: x * x
print(square(5))
```

Use cases:

- Short computations
- Used with map( ), filter( ), reduce( )

Filter, Map, and Reduce

These functions enable functional programming in Python.

map( )

Applies a function to all elements.

```
numbers = [1, 2, 3]
result = list(map(lambda x: x * 2, numbers))
```

filter( )

Filters elements based on a condition.

PYTHON

```
even = list(filter(lambda x: x % 2 == 0, numbers))
```

reduce( )

Reduces elements to a single value.

```
from functools import reduce
total = reduce(lambda x, y: x + y, numbers)
```

Pickling

Pickling is the process of **serializing Python objects** into files.

Example

```
import pickle

data = {"name": "Python", "version": 3}
with open("data.pkl", "wb") as file:
    pickle.dump(data, file)
```

Used for object persistence and data storage.

Python JSON

JSON is widely used for **data exchange** between applications.

JSON Serialization

```
import json

data = {"name": "Alice", "age": 25}
json_data = json.dumps(data)
```

JSON Deserialization

```
data = json.loads(json_data)
```

PYTHON

Python API and Web Scraping

Python supports API consumption and web scraping using libraries like:

- requests
- BeautifulSoup
- urllib

API Example

```
import requests

response = requests.get("https://api.example.com/data")
print(response.json())
```

Working with Python SQLite

SQLite is a lightweight relational database.

Database Example

```
import sqlite3

conn = sqlite3.connect("data.db")
cursor = conn.cursor()
cursor.execute("CREATE TABLE IF NOT EXISTS users(id INTEGER,
name TEXT)")
conn.commit()
conn.close()
```

Iterators and Generators

Iterators allow traversal of collections, while generators yield values one at a time.

Generator Example

```
def count_up(n):
    for i in range(n):
        yield i
```

Generators improve memory efficiency.

PYTHON

Decorators

Decorators modify function behavior without changing its code.

Example

```
def my_decorator(func):
    def wrapper():
        print("Before function")
        func()
        print("After function")
    return wrapper

@my_decorator
def say_hello():
    print("Hello")
```

Regular Expressions

Regular expressions enable pattern matching in strings.

Example

```
import re

pattern = r"\d+"
result = re.findall(pattern, "Order 123 delivered")
Used in validation, searching, and text processing.
```

Virtual Environments and Package Management

Virtual environments isolate project dependencies.

Creating a Virtual Environment

```
python -m venv env
```

Installing Packages

PYTHON

```
pip install requests
```

Advanced Python topics enhance code efficiency, reusability, and maintainability. These concepts are essential for building enterprise-level applications, automation systems, APIs, and data-driven solutions.

PYTHON

13. Harnessing Generative Artificial Intelligence (GenAI) and GitHub Copilot

Introduction

Generative Artificial Intelligence (GenAI) refers to AI models that can generate content such as code, text, images, and other data. GitHub Copilot, powered by AI, is a coding assistant that leverages GenAI to help developers write code faster, improve code quality, and explore alternative implementations.

Using GitHub Copilot effectively accelerates software development, aids in learning, and enhances productivity by suggesting code snippets, documentation, and debugging assistance.

GitHub Copilot Overview

GitHub Copilot is an AI-powered code completion tool integrated into popular IDEs like Visual Studio Code, JetBrains, and Neovim. It predicts and suggests code based on:

- Existing project code
- Programming language syntax
- Context from comments and function names

Basic Copilot Usage

- Start typing code or a comment describing functionality.
- Copilot suggests one or more code completions.
- Accept suggestions with Tab or Enter.
- Modify and refine suggestions as needed.

Understanding Copilot Suggestions

- Suggestions may include complete functions, loops, or small snippets.
- Copilot evaluates context to generate relevant code.
- Review suggestions carefully to ensure correctness.

Generative Commit Messages

- Copilot can generate meaningful commit messages.
- Helps maintain professional, descriptive version control history.
- **Example:**

PYTHON

"Refactored data processing function for improved performance"

Copilot Chat and CLI

- Copilot Chat allows interactive Q&A for coding guidance within IDE.
- CLI integration enables AI-powered suggestions via command line tools.

Copilot for Code Optimization and Refactoring

- Automatically suggests optimized solutions.
- Refactoring recommendations improve readability and maintainability.
- Supports best practices in Python and other languages.

Copilot's Strengths and Limitations

Strengths:

- Accelerates development
- Reduces repetitive coding
- Provides learning support for new languages

Limitations:

- Suggestions may contain errors
- Not a replacement for human review
- Limited understanding of project-specific business logic

Troubleshooting and Common Issues

- Suggestions not appearing → Check IDE plugin installation
- Outdated models → Update Copilot plugin
- Context mismatch → Add descriptive comments

GitHub Copilot Language Support

- Python, JavaScript, TypeScript, Java, C#, Go, and more
- Copilot adapts suggestions to language syntax and common libraries

PYTHON

Code Documentation with Copilot

- Generates inline comments and docstrings.
- Helps maintain readable and maintainable code.

Copilot Best Practices

- Use suggestions as guidance, not final code.
- Combine Copilot with unit testing.
- Refactor and review generated code.
- Integrate Copilot in team development workflow carefully.

NLP and GitHub Copilot

Copilot leverages Natural Language Processing (NLP) to interpret comments and descriptive instructions, transforming them into functional code.

Harnessing GenAI through GitHub Copilot enables developers to write code efficiently, document effectively, and enhance learning. While Copilot is a powerful assistant, responsible usage, careful review, and testing remain crucial for delivering high-quality software.

PYTHON

14. Mini Project Development

Introduction

Mini projects are practical implementations that consolidate learning and demonstrate the application of Python programming concepts. They provide hands-on experience in problem-solving, coding, debugging, and documentation. Mini projects are essential for students and developers to bridge theoretical knowledge with real-world applications.

This section guides the process of designing, developing, and presenting Python mini projects.

Objectives of Mini Projects

- Apply Python programming concepts learned in theory.
- Practice modular coding, data structures, and algorithms.
- Gain experience in file handling, libraries, and debugging.
- Develop problem-solving and analytical thinking skills.
- Prepare for real-world software development projects.

Project Development Lifecycle

1. Problem Identification

- Select a problem that is simple yet meaningful.
- Define clear project goals and outcomes.
- Example: Temperature Monitoring System, Mini Chat Application, or Student Management System.

2. Requirements Analysis

- Identify software requirements:
    1. Python version and IDE
    2. Required libraries (e.g., Pandas, Matplotlib)
    3. Input/output devices (if any)
- Determine data structures and algorithms needed.

3. System Design

- Break the project into modules and functions.

PYTHON

- Prepare flowcharts or pseudocode.
- Define classes and objects if using OOP.

4. Coding Implementation

- Write modular, readable, and commented code.
- Use appropriate data structures, libraries, and functions.
- Include error handling and debugging mechanisms.

5. Testing

- Conduct unit tests for each module.
- Perform integration testing for overall functionality.
- Handle edge cases and invalid inputs.

6. Documentation

- Prepare a project report including:
  - Problem statement and objectives
  - System design and flowcharts
  - Code snippets
  - Test results and screenshots
  - Conclusion and future enhancements

Example Mini Project Ideas

1. **Temperature Monitoring System**
   - Input: Temperature readings from sensors
   - Output: Display current temperature and alerts for critical values
   - Libraries: Matplotlib for visualization, Pandas for data logging
2. **Mini Chat Application**
   - Features: Send and receive messages over a local network
   - Python Libraries: socket, threading, tkinter for GUI
3. **Student Management System**

- Features: Add, update, delete student records
- File handling: Store records in CSV or JSON files
- Error handling for invalid inputs

4. **Calculator Application**
    - Features: Perform basic arithmetic operations
    - GUI implementation with Tkinter
    - Unit testing for all operations
5. **To-Do List Application**
    - Features: Add, remove, mark tasks as completed
    - Persistent storage using JSON or SQLite

Best Practices for Mini Project Development

- Keep projects simple and focused on learning objectives.
- Write clean, modular, and well-documented code.
- Follow proper naming conventions for variables and functions.
- Use version control (e.g., Git) to track code changes.
- Test thoroughly and handle exceptions properly.
- Include a user-friendly interface if applicable.

Learning Outcomes

By developing mini projects, learners:

- Strengthen their Python programming and problem-solving skills.
- Gain hands-on experience with Python libraries, OOP, and file handling.
- Learn to debug and optimize code.
- Acquire the ability to document and present projects professionally.

Mini project development is a vital part of Python learning, consolidating theory into practice. It prepares learners for complex real-world projects, enhances confidence, and provides a strong foundation for future software development or research applications.